

Universidad Nacional de Colombia

Facultad de Ingeniería
Departamento de Sistemas e Industrial
Ingeniería de Software I

Hola Mundo

Java · Spring Boot · PostgreSQL

Tutorial paso a paso

Integrantes: Julian Andrés Foglia Wilches
Laura Valentina Hernandez Torres
Eduard Joel Ostos Castro
Santiago Cubides

Profesor: Oscar Eduardo Álvarez Rodríguez

Fecha: 13 de mayo de 2026

Índice

1. Introducción	3
2. Tecnologías y herramientas	3
2.1. Stack seleccionado	3
2.2. ¿Qué es cada herramienta?	3
3. Prerrequisitos e instalación	4
3.1. Java 17 (JDK)	4
3.2. Docker y Docker Compose	4
3.3. Maven (Wrapper incluido)	4
3.4. Editor de código	5
4. Crear el proyecto con Spring Initializr	5
4.1. Paso 1 — Abrir Spring Initializr	5
4.2. Paso 2 — Configurar los metadatos	5
4.3. Paso 3 — Agregar dependencias	6
4.4. Paso 4 — Generar y descargar	6
4.5. Paso 5 — Verificar la estructura inicial	6
5. Arquitectura de la aplicación	7
6. Levantar PostgreSQL con Docker	8
6.1. Crear el archivo docker-compose.yml	8
6.2. Iniciar el contenedor	9
7. Configurar Spring Boot (application.yml)	10
8. Implementar las capas del backend	11
8.1. Entidad: Mensaje.java	11
8.2. Repositorio: MensajeRepository.java	12
8.3. Servicio: MensajeService.java	13
8.4. Controlador: MensajeController.java	14
9. Insertar el dato inicial (semilla)	15
9.1. Opción A — DataLoader.java (recomendada)	15
9.2. Opción B — data.sql	16
10. Ejecutar la aplicación	16
10.1. Paso 1 — Verificar que PostgreSQL está corriendo	16
10.2. Paso 2 — Iniciar Spring Boot	16
11. Probar desde Postman	17
11.1. Conceptos REST rápidos	18
11.2. Prueba GET /hola desde Postman	18
11.3. Prueba POST /hola desde Postman (opcional)	18
11.4. Prueba alternativa con curl	18
12. Verificar los datos en PostgreSQL	19

13.Estructura final del proyecto	19
14.Resumen del flujo completo	20
Conclusiones	20

1. Introducción

Este tutorial te guía paso a paso en la construcción de una aplicación **backend** con Java y Spring Boot que se conecta a una base de datos PostgreSQL.

Al finalizar tendrás:

- Un contenedor Docker con PostgreSQL corriendo.
- Una entidad **Mensaje** mapeada con JPA/Hibernate.
- Las capas de *Repositorio*, *Servicio* y *Controlador* implementadas y comunicadas.
- Un endpoint verificado desde Postman que retorna `{id:1,"texto":"Hola Mundo"}`.

2. Tecnologías y herramientas

2.1. Stack seleccionado

Rol	Tecnología	Versión
Lenguaje	Java	17
Framework	Spring Boot	3.x
ORM	Spring Data JPA / Hibernate	incluido
Base de datos	PostgreSQL	14+
Gestor de build	Maven	3.9+
Contenedores	Docker + Docker Compose	24+

2.2. ¿Qué es cada herramienta?

Java 17 Lenguaje de programación fuertemente tipado, orientado a objetos y multi-plataforma. Es el lenguaje principal de nuestra aplicación.

Spring Boot

Framework que simplifica el desarrollo de aplicaciones web en Java. Incluye un servidor web embebido (Tomcat), auto-configuración y un ecosistema de librerías listo para usar.

Spring Data JPA / Hibernate

ORM (Object-Relational Mapping) que traduce clases Java a tablas SQL y viceversa. Con `JpaRepository` obtenemos operaciones CRUD sin escribir una sola línea de SQL.

PostgreSQL

Sistema de gestión de bases de datos relacional, robusto y de código abierto. Almacenará nuestros mensajes.

Docker / Docker Compose

Permiten levantar PostgreSQL en un contenedor aislado con un solo comando, sin instalar el servidor de base de datos directamente en el equipo.

3. Prerrequisitos e instalación

3.1. Java 17 (JDK)

Verifica la instalación abriendo una terminal:

```
java -version
```

La salida esperada es similar a:

```
java version "17.0.10" 2024-01-16 LTS
Java(TM) SE Runtime Environment (build 17.0.10+11-LTS-240)
```

Si no tienes Java instalado, descárgalo desde:

<https://www.oracle.com/java/technologies/javase/jdk17-archive-downloads.html>

```
C:\Users\Negro>java --version
java 17.0.10 2024-01-16 LTS
Java(TM) SE Runtime Environment (build 17.0.10+11-LTS-240)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.10+11-LTS-240, mixed mode, sharing)
```

Figura 1: Salida del comando `java -version` en terminal

3.2. Docker y Docker Compose

```
docker --version          # Docker version 24.x.x
docker compose version    # Docker Compose version 2.x.x
```

Descarga e instalación:

- **Windows / macOS:** Docker Desktop en <https://www.docker.com/products/docker-desktop>
- **Linux:** <https://docs.docker.com/engine/install/>

```
C:\Users\Negro>docker --version
Docker version 29.4.3, build 055a478

C:\Users\Negro>docker compose version
Docker Compose version v5.1.3
```

Figura 2: Verificación de Docker y Docker Compose en terminal

3.3. Maven (Wrapper incluido)

El proyecto generado por Spring Initializr incluye un *Maven Wrapper* (`mvnw` / `mvnw.cmd`), por lo que **no es necesario instalar Maven globalmente**. De todas formas, si lo deseas:

<https://maven.apache.org>

3.4. Editor de código

Cualquiera de los siguientes funciona bien:

- **IntelliJ IDEA Community:** <https://www.jetbrains.com/idea/download/>
- **Visual Studio Code:** <https://code.visualstudio.com/>

4. Crear el proyecto con Spring Initializr

4.1. Paso 1 — Abrir Spring Initializr

Ingresa a <https://start.spring.io> en tu navegador.

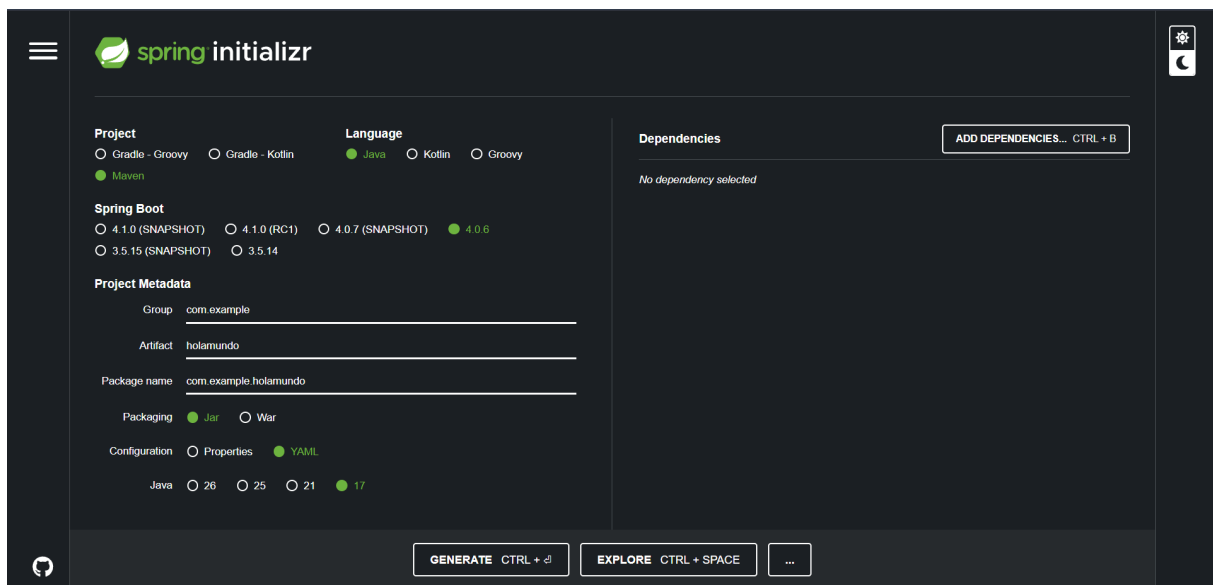


Figura 3: Pantalla principal de Spring Initializr

4.2. Paso 2 — Configurar los metadatos

Llena el formulario con los siguientes valores:

Campo	Valor	Descripción
Project	Maven	Gestión de dependencias y compilación
Language	Java	Lenguaje principal
Spring Boot	3.x (estable)	Versión del framework
Group	com.ejemplo	Paquete base del proyecto
Artifact	holamundo	Nombre del proyecto
Packaging	Jar	Formato del ejecutable
Java	17	Versión de Java

4.3. Paso 3 — Agregar dependencias

Haz clic en **ADD DEPENDENCIES** y selecciona:

- **Spring Web** — para crear controladores REST.
- **Spring Data JPA** — para el ORM con Hibernate.
- **PostgreSQL Driver** — conector JDBC para PostgreSQL.

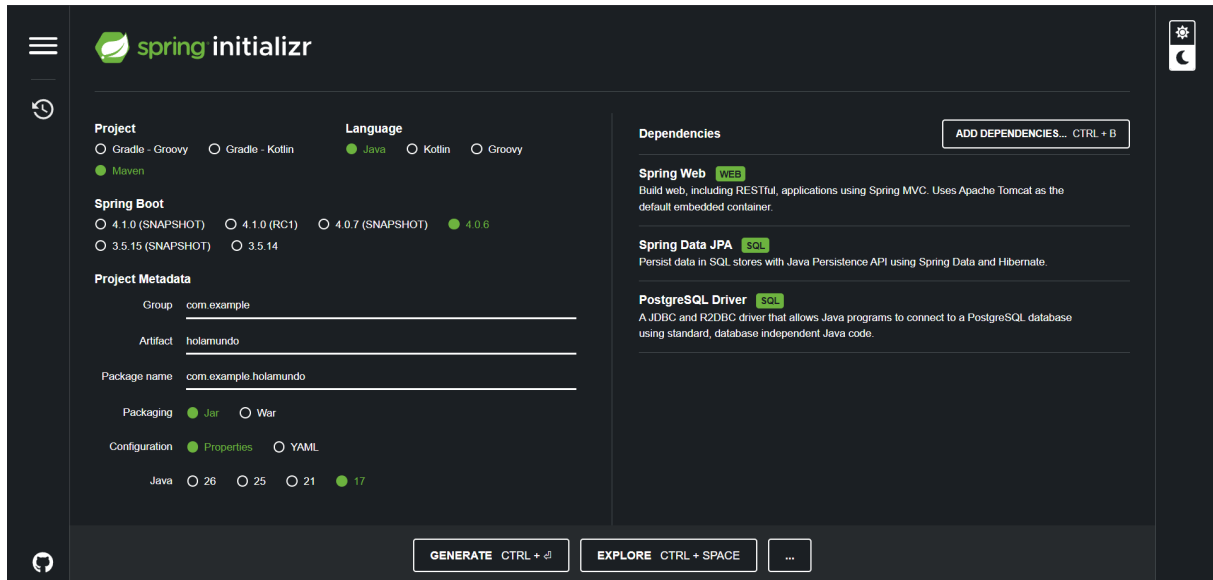


Figura 4: Dependencias seleccionadas

4.4. Paso 4 — Generar y descargar

Haz clic en **GENERATE**. Se descargará un archivo ZIP. Extráelo en una carpeta de trabajo, por ejemplo:

```
# Linux / macOS
~/proyectos/holamundo

# Windows
C:\Users\TuNombre\proyectos\holamundo
```

4.5. Paso 5 — Verificar la estructura inicial

Abre la carpeta en tu editor. La estructura inicial del proyecto es:

```
holamundo/
├── .mvn/wrapper/maven-wrapper.properties
├── src/
│   └── main/
│       ├── java/com/ejemplo/holamundo/
│       │   └── HolamundoApplication.java
│       ├── resources/
│       │   └── application.properties
```

```

test/
    java/com/ejemplo/holamundo/
        HolamundoApplicationTests.java

mvnw
mvnw.cmd
pom.xml
    
```

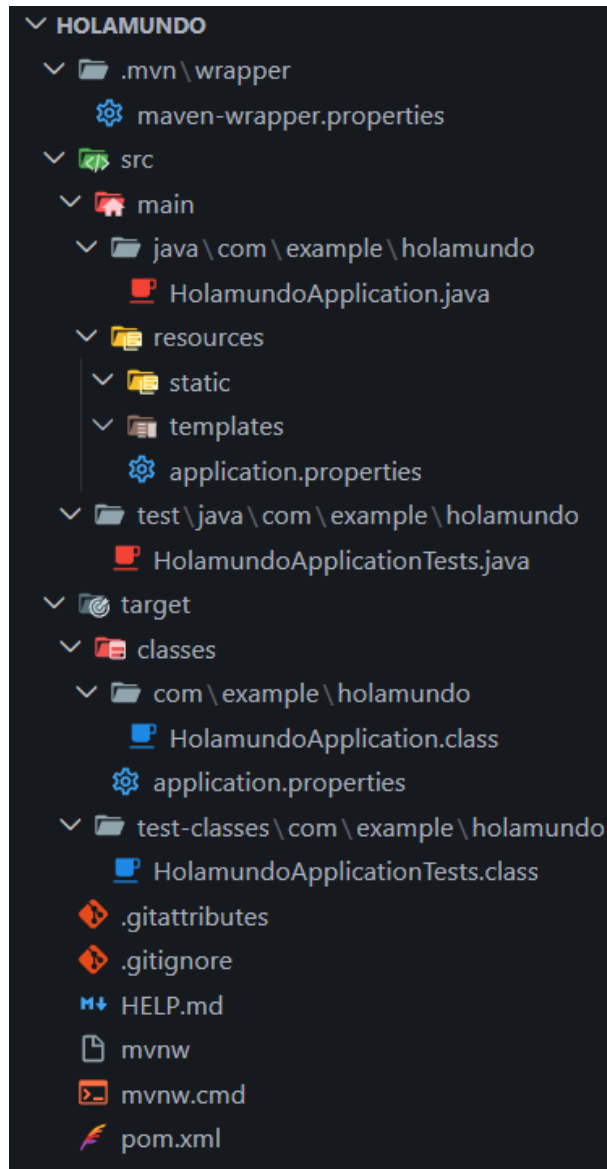


Figura 5: Estructura del proyecto abierta

5. Arquitectura de la aplicación

La aplicación sigue la arquitectura de **tres capas** estándar de Spring Boot:

Capa	Clase	Responsabilidad
Controlador	MensajeController.java	Recibe peticiones HTTP, devuelve JSON
Servicio	MensajeService.java	Lógica de negocio
Repositorio	MensajeRepository.java	Acceso a la base de datos (JPA)
Entidad / Modelo	Mensaje.java	Representa la tabla en PostgreSQL

Flujo de una petición GET /hola:

Cliente (Postman) → Controller → Service → Repository → PostgreSQL →
(respuesta de vuelta por el mismo camino)

La estructura final de carpetas del proyecto será:

```

holamundo/
  pom.xml
  src/main/
    java/com/ejemplo/holamundo/
      HolamundoApplication.java
    model/
      Mensaje.java                      Entidad JPA
    repository/
      MensajeRepository.java
    service/
      MensajeService.java
    controller/
      MensajeController.java
    config/
      DataLoader.java                  Datos iniciales
  resources/
    application.yml

```

6. Levantar PostgreSQL con Docker

6.1. Crear el archivo docker-compose.yml

En la raíz del proyecto (junto a pom.xml), crea el archivo docker-compose.yml con el siguiente contenido:

```

1 version: '3.8'
2
3 services:
4   db:
5     image: postgres:15
6     container_name: holamundo-db
7     environment:
8       POSTGRES_DB: hola_mundo
9       POSTGRES_USER: hm_user
10      POSTGRES_PASSWORD: hm_pass
11     ports:

```

```

12     - "5432:5432"
13     volumes:
14     - db-data:/var/lib/postgresql/data
15     restart: unless-stopped
16
17 volumes:
18     db-data:

```

Listing 1: docker-compose.yml

Explicación de las variables:

- POSTGRES_DB — nombre de la base de datos.
- POSTGRES_USER — usuario de conexión.
- POSTGRES_PASSWORD — contraseña de conexión.
- volumes — persiste los datos aunque el contenedor se detenga.

6.2. Iniciar el contenedor

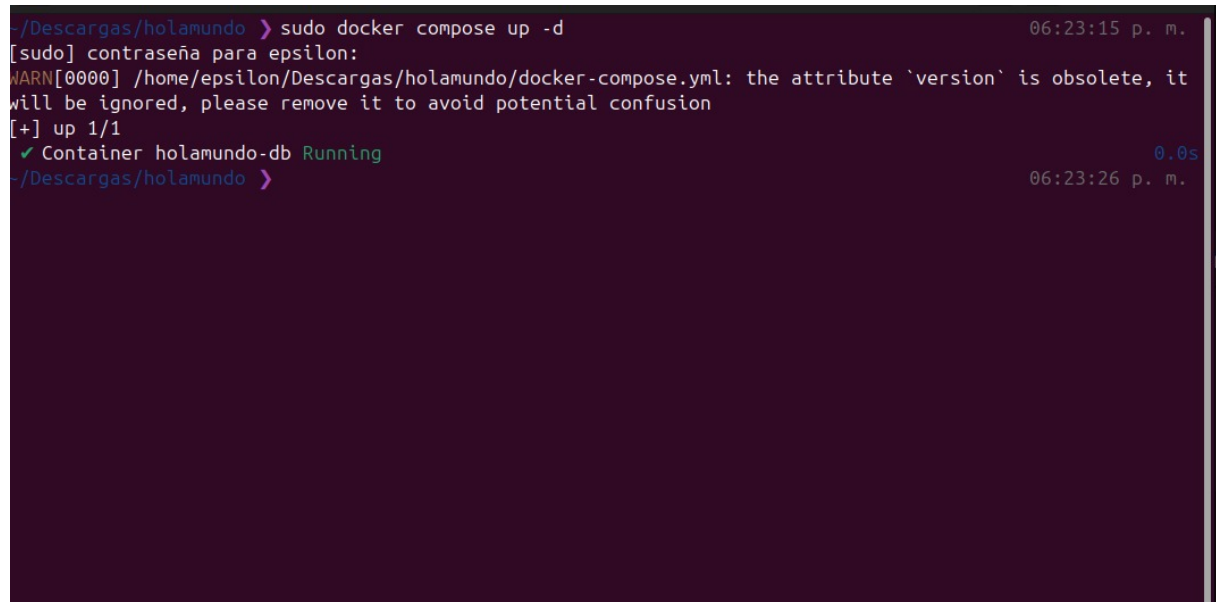
Abre una terminal en la carpeta del proyecto y ejecuta:

```

# Linux / macOS (puede requerir sudo en Linux)
docker compose up -d

# Verificar que el contenedor esta corriendo
docker ps

```



```

~/Descargas/holamundo > sudo docker compose up -d                                06:23:15 p. m.
[sudo] contraseña para epsilon:
WARN[0000] /home/epsilon/Descargas/holamundo/docker-compose.yml: the attribute `version` is obsolete, it
will be ignored, please remove it to avoid potential confusion
[+] up 1/1
✓ Container holamundo-db Running                                                    0.0s
~/Descargas/holamundo >

```

Figura 6: Salida de docker ps confirmando el contenedor holamundo-db

7. Configurar Spring Boot (application.yml)

Elimina (o vacía) el archivo `application.properties` y crea `application.yml` en la misma ruta `src/main/resources/application.yml`:

```
1 server:
2   port: 8080
3
4 spring:
5   datasource:
6     url: jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/${
7       {DB_NAME:hola_mundo}
8     username: ${DB_USER:hm_user}
9     password: ${DB_PASS:hm_pass}
10    driver-class-name: org.postgresql.Driver
11  jpa:
12    hibernate:
13      ddl-auto: update          # Crea/actualiza tablas
14      autom ticamente
15    show-sql: true             # Muestra consultas SQL en consola
16    properties:
17      hibernate:
18        format_sql: true
19  sql:
20    init:
21      mode: always
```

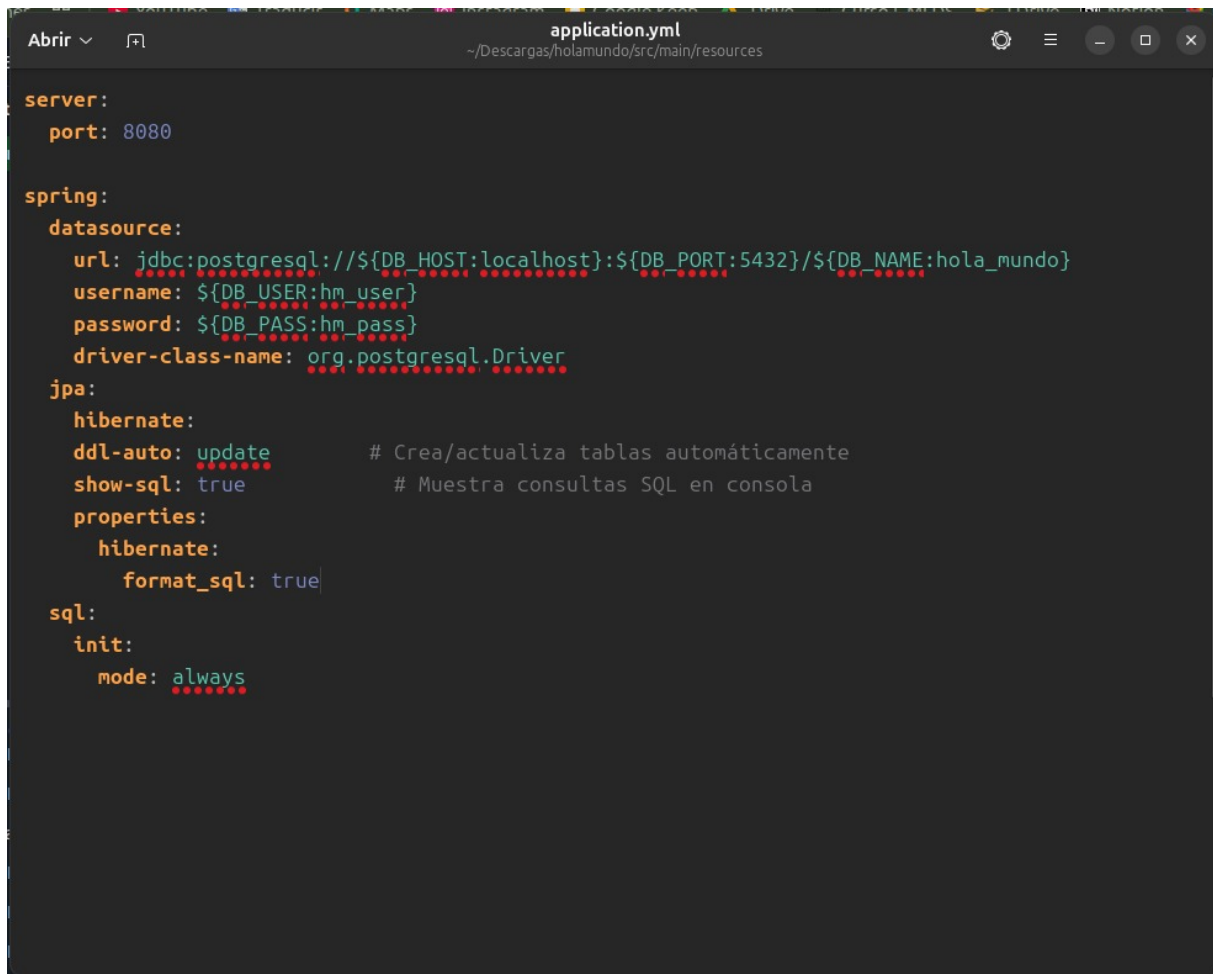
Listing 2: `src/main/resources/application.yml`

Puntos clave:

ddl-auto: update Hibernate crea o actualiza las tablas al iniciar la aplicación, basándose en las clases anotadas con `@Entity`. Ideal para desarrollo.

show-sql: true Imprime en consola cada consulta SQL ejecutada, útil para depuración.

Variables de entorno La sintaxis `${VAR:valor_por_defecto}` permite sobrescribir la configuración en distintos entornos sin modificar el archivo.



```

server:
  port: 8080

spring:
  datasource:
    url: jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/${DB_NAME:hola_mundo}
    username: ${DB_USER:hm_user}
    password: ${DB_PASS:hm_pass}
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: update          # Crea/actualiza tablas automáticamente
      show-sql: true           # Muestra consultas SQL en consola
    properties:
      hibernate:
        format_sql: true
  sql:
    init:
      mode: always
    
```

Figura 7: Archivo application.yml abierto en el IDE

8. Implementar las capas del backend

8.1. Entidad: Mensaje.java

Crea la carpeta model/ y dentro el archivo src/main/java/com/ejemplo/holamundo/model/Mensaje.java

```

1 package com.ejemplo.holamundo.model;
2
3 import jakarta.persistence.*;
4
5 /**
6  * Entidad JPA que representa la tabla "mensaje" en PostgreSQL.
7  * Cada instancia de esta clase equivale a una fila de la tabla.
8  */
9 @Entity
10 @Table(name = "mensaje")
11 public class Mensaje {
12
13     /** Clave primaria auto-incremental gestionada por PostgreSQL
14     */
    
```

```

14     @Id
15     @GeneratedValue(strategy = GenerationType.IDENTITY)
16     private Long id;
17
18     /** Texto del mensaje. No puede ser nulo. */
19     @Column(nullable = false)
20     private String texto;
21
22     // Constructor vacío requerido por JPA
23     public Mensaje() {}
24
25     public Mensaje(String texto) {
26         this.texto = texto;
27     }
28
29     // Getters y setters
30     public Long getId()           { return id; }
31     public String getTexto()      { return texto; }
32     public void setTexto(String t) { this.texto = t; }
33 }

```

Listing 3: model/Mensaje.java

Anotaciones importantes:

@Entity Indica a JPA que esta clase es una tabla de BD.

@Table(name = "mensaje") Nombre explícito de la tabla en PostgreSQL.

@Id Marca la clave primaria.

@GeneratedValue(IDENTITY) El id lo asigna automáticamente la BD (autoincremental).

@Column(nullable = false) El campo texto no puede ser nulo.

8.2. Repositorio: MensajeRepository.java

Crea la carpeta repository/ y el archivo src/main/java/com/ejemplo/holamundo/repository/MensajeRepository.java

```

1 package com.ejemplo.holamundo.repository;
2
3 import com.ejemplo.holamundo.model.Mensaje;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 /**
8  * Repositorio JPA para la entidad Mensaje.
9  *
10  * Al extender JpaRepository obtenemos gratis:
11  *   save(), findAll(), findById(), deleteById(), count(), etc.
12  * Sin escribir ninguna consulta SQL.
13  */
14 @Repository
15 public interface MensajeRepository extends JpaRepository<Mensaje,
    Long> {

```

```

16      // No se necesita código adicional para operaciones CRUD bá
      sicas
17  }

```

Listing 4: repository/MensajeRepository.java

8.3. Servicio: MensajeService.java

Crea la carpeta `service/` y el archivo `src/main/java/com/ejemplo/holamundo/service/MensajeServ`

```

1  package com.ejemplo.holamundo.service;
2
3  import com.ejemplo.holamundo.model.Mensaje;
4  import com.ejemplo.holamundo.repository.MensajeRepository;
5  import org.springframework.stereotype.Service;
6
7  import java.util.List;
8
9  /**
10   * Capa de servicio: contiene la logica de negocio.
11   * Actua de intermediario entre el Controlador y el Repositorio.
12   */
13  @Service
14  public class MensajeService {
15
16      // Inyeccion por constructor (practica recomendada sobre
17      // @Autowired)
18      private final MensajeRepository repository;
19
20      public MensajeService(MensajeRepository repository) {
21          this.repository = repository;
22      }
23
24      /** Devuelve todos los mensajes almacenados. */
25      public List<Mensaje> findAll() {
26          return repository.findAll();
27      }
28
29      /** Guarda un nuevo mensaje en la BD. */
30      public Mensaje save(Mensaje m) {
31          return repository.save(m);
32      }
33
34      /**
35       * Devuelve el primer mensaje encontrado, o null si no existe
36       * ninguno.
37       * Es el que usaremos en el endpoint GET /hola.
38       */
39      public Mensaje findFirst() {
40          return repository.findAll()
41              .stream()

```

```

40         .findFirst()
41         .orElse(null);
42     }
43 }

```

Listing 5: service/MensajeService.java

8.4. Controlador: MensajeController.java

Crea la carpeta controller/ y el archivo src/main/java/com/ejemplo/holamundo/controller/Mensa

```

1  package com.ejemplo.holamundo.controller;
2
3  import com.ejemplo.holamundo.model.Mensaje;
4  import com.ejemplo.holamundo.service.MensajeService;
5  import org.springframework.http.ResponseEntity;
6  import org.springframework.web.bind.annotation.*;
7
8  /**
9   * Controlador REST que expone los endpoints de la entidad
10   * Mensaje.
11   *
12   * GET /hola      devuelve el primer mensaje (o 404 si no
13   *               existe)
14   * POST /hola     crea un nuevo mensaje
15   */
16  @RestController
17  @RequestMapping("/hola")
18  public class MensajeController {
19
20      private final MensajeService service;
21
22      public MensajeController(MensajeService service) {
23          this.service = service;
24      }
25
26      /**
27       * GET /hola
28       * Retorna el primer mensaje de la BD en formato JSON.
29       * Código 200 OK si existe.
30       * Código 404 Not Found si la tabla esta vacia.
31       */
32      @GetMapping
33      public ResponseEntity<Mensaje> getHola() {
34          Mensaje m = service.findFirst();
35          if (m == null) {
36              return ResponseEntity.notFound().build();    // 404
37          }
38          return ResponseEntity.ok(m);                    // 200
39      }
40
41  }

```

```

39  /**
40   * POST /hola
41   * Crea un nuevo mensaje a partir del cuerpo JSON de la
42   * petición.
43   * Retorna 201 Created con el objeto guardado.
44   */
45  @PostMapping
46  public ResponseEntity<Mensaje> createHola(@RequestBody
47      Mensaje nuevo) {
48      Mensaje saved = service.save(nuevo);
49      return ResponseEntity.status(201).body(saved); // 201
50  }

```

Listing 6: controller/MensajeController.java

9. Insertar el dato inicial (semilla)

La base de datos arranca vacía. Hay dos formas de insertar el mensaje “Hola Mundo” automáticamente al iniciar la aplicación.

9.1. Opción A — DataLoader.java (recomendada)

Crea la carpeta config/ y el archivo src/main/java/com/ejemplo/holamundo/config/DataLoader.java

```

1  package com.ejemplo.holamundo.config;
2
3  import com.ejemplo.holamundo.model.Mensaje;
4  import com.ejemplo.holamundo.repository.MensajeRepository;
5  import org.springframework.boot.CommandLineRunner;
6  import org.springframework.context.annotation.Bean;
7  import org.springframework.context.annotation.Configuration;
8
9  /**
10   * Clase de configuracion que inserta datos de prueba al arrancar
11   * .
12   * Solo inserta si la tabla esta vacia (evita duplicados en
13   * reinicios).
14   */
15  @Configuration
16  public class DataLoader {
17
18      @Bean
19      CommandLineRunner initDatabase(MensajeRepository repo) {
20          return args -> {
21              if (repo.count() == 0) {
22                  repo.save(new Mensaje("Hola Mundo"));
23                  System.out.println(">>> Dato inicial insertado:
24                      Hola Mundo");
25              }
26          };
27      }
28  }

```



```

22         } else {
23             System.out.println(">>> La tabla ya tiene datos,
                                no se inserta nada.");
24         }
25     };
26 }
27 }

```

Listing 7: config/DataLoader.java

9.2. Opción B — data.sql

Como alternativa, crea el archivo `src/main/resources/data.sql`:

```

1 INSERT INTO mensaje (texto) VALUES ('Hola Mundo');

```

Listing 8: data.sql

Nota

Si usas `data.sql` junto con `DataLoader.java`, el mensaje se insertará dos veces. Usa **uno u otro**, no ambos.

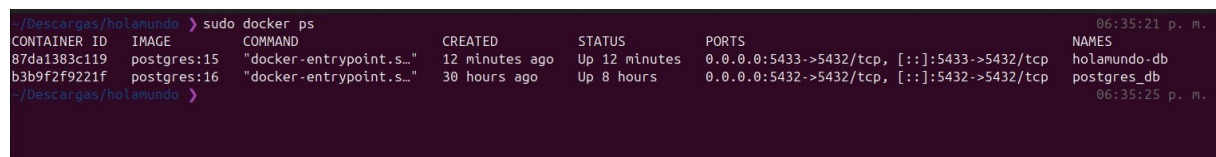
10. Ejecutar la aplicación

10.1. Paso 1 — Verificar que PostgreSQL está corriendo

```
docker ps
```

Debes ver el contenedor `holamundo-db` con estado `Up`. Si no aparece, reinícialo:

```
docker compose up -d
```



```

~/Descargas/holamundo > sudo docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                                                                 NAMES
87da1383c119   postgres:15 "docker-entrypoint.s..." 12 minutes ago Up 12 minutes 0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp   holamundo-db
b3b9f2f9221f   postgres:16 "docker-entrypoint.s..." 30 hours ago   Up 8 hours    0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp   postgres_db
~/Descargas/holamundo >

```

Figura 8: Salida de `docker ps` mostrando el contenedor en estado `Up`

10.2. Paso 2 — Iniciar Spring Boot

Abre una nueva terminal en la carpeta raíz del proyecto y ejecuta:

```

# Linux / macOS
./mvnw spring-boot:run

```

```
# Windows
mvnw.cmd spring-boot:run
```

La primera ejecución descargará todas las dependencias (puede tardar 1–2 minutos). En las siguientes será mucho más rápida.

Cuando la aplicación esté lista verás en consola algo como:

```

      .
     /\ \ / - - - - , _ _ _ _ _ ( _ ) _ _ _ _ _ \ \ \ \ \ \
    ...
:: Spring Boot ::                                (v3.x.x)

... Started HolamundoApplication in 3.2 seconds
... Tomcat started on port(s): 8080 (http)
>>> Dato inicial insertado: Hola Mundo

```

```

epsilon@epsilon-ideaPad-Slim-3-151RH10:~/Descargas/holamundo
~/Descargas/holamundo > chmod +x mvnw
~/Descargas/holamundo > ./mvnw spring-boot:run
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:holamundo >-----
[INFO] Building 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] >>> spring-boot:4.0.6:run (default-cli) > test-compile @ holamundo >>>
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ holamundo ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.14.1:compile (default-compile) @ holamundo ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 6 source files with javac [debug parameters release 17] to target/classes
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ holamundo ---
[INFO] skip non existing resourceDirectory /home/epsilon/Descargas/holamundo/src/test/resources
[INFO]
[INFO] --- compiler:3.14.1:testCompile (default-testCompile) @ holamundo ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 1 source file with javac [debug parameters release 17] to target/test-classes
[INFO]
[INFO] <<< spring-boot:4.0.6:run (default-cli) < test-compile @ holamundo <<<
[INFO]
[INFO]
[INFO] --- spring-boot:4.0.6:run (default-cli) @ holamundo ---
[INFO] Attaching agents: []

      .
     /\ \ / - - - - , _ _ _ _ _ ( _ ) _ _ _ _ _ \ \ \ \ \ \
    ...
:: Spring Boot ::                                (v4.0.6)

2026-05-13T18:42:29.655-05:00 INFO 49126 --- [main] c.e.holamundo.HolamundoApplication : Starting HolamundoApplication using Java 17.0.18 with PID 49126 (/home/epsilon/Descargas/holamundo/target/classes started by epsilon in /home/epsilon/Descargas/holamundo)
2026-05-13T18:42:29.658-05:00 INFO 49126 --- [main] c.e.holamundo.HolamundoApplication : No active profile set, falling back to 1 default profile: "default"
2026-05-13T18:42:30.169-05:00 INFO 49126 --- [main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-05-13T18:42:30.191-05:00 INFO 49126 --- [main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 12 ms. Found 0 JPA repository interface
s.
2026-05-13T18:42:30.598-05:00 INFO 49126 --- [main] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-05-13T18:42:30.615-05:00 INFO 49126 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]

```

Figura 9: Consola mostrando el arranque exitoso de Spring Boot en el puerto 8080

11. Probar desde Postman

11.1. Conceptos REST rápidos

Concepto	Descripción	Ejemplo
Endpoint	URL que atiende peticiones	/hola
GET	Solicita información sin modificar nada	Leer el mensaje
POST	Crea un nuevo recurso	Guardar nuevo mensaje
JSON	Formato de intercambio de datos	{id:1,"texto":"Hola"}
200 OK	La solicitud fue exitosa	Mensaje encontrado
404 Not Found	Recurso no encontrado	Tabla vacía
201 Created	Recurso creado exitosamente	Mensaje guardado

11.2. Prueba GET /hola desde Postman

1. Abre Postman y crea una nueva petición.
2. Selecciona el método **GET**.
3. Escribe la URL: `http://localhost:8080/hola`
4. Haz clic en **Send**.

Respuesta esperada (HTTP 200):

```

1 {
2   "id": 1,
3   "texto": "Hola Mundo"
4 }
```

11.3. Prueba POST /hola desde Postman (opcional)

1. Método: **POST**.
2. URL: `http://localhost:8080/hola`
3. En la pestaña **Body** → **raw** → tipo **JSON**.
4. Escribe el cuerpo:

```

1 {
2   "texto": "Mi nuevo mensaje"
3 }
```

Respuesta esperada (HTTP 201):

```

1 {
2   "id": 2,
3   "texto": "Mi nuevo mensaje"
4 }
```

11.4. Prueba alternativa con curl

Si prefieres la línea de comandos:

```
# GET
curl -i http://localhost:8080/hola

# POST
curl -i -X POST http://localhost:8080/hola \
  -H "Content-Type: application/json" \
  -d '{"texto": "Hola desde curl"}'
```

12. Verificar los datos en PostgreSQL

Puedes conectarte al contenedor y verificar los datos directamente:

```
# Conectarse al contenedor
docker exec -it holamundo-db psql -U hm_user -d hola_mundo

# Dentro de psql:
\dt                                -- listar tablas
SELECT * FROM mensaje;            -- ver todos los mensajes
\q                                 -- salir
```

13. Estructura final del proyecto

```
holamundo/
├── docker-compose.yml           Base de datos
├── en Docker
│   ├── pom.xml                 Dependencias
│   └── Maven
│       ├── src/
│       │   ├── main/
│       │   │   ├── java/com/ejemplo/holamundo/
│       │   │   │   ├── HolamundoApplication.java
│       │   │   │   ├── config/
│       │   │   │   │   ├── DataLoader.java           Semilla
│       │   │   │   └── inicial
│       │   │       ├── controller/
│       │   │       │   ├── MensajeController.java
│       │   │       ├── model/
│       │   │       │   ├── Mensaje.java
│       │   │       ├── repository/
│       │   │       │   ├── MensajeRepository.java
│       │   │       ├── service/
│       │   │       │   ├── MensajeService.java
│       │   └── resources/
│       │       └── application.yml
```

14. Resumen del flujo completo

1. **Docker** levanta PostgreSQL en el puerto 5432 con la base de datos `hola_mundo`.
2. Spring Boot arranca, lee `application.yml` y se conecta a PostgreSQL.
3. **Hibernate** lee la clase `Mensaje.java` y crea (o actualiza) automáticamente la tabla `mensaje`.
4. **DataLoader** inserta el registro "Hola Mundo" si la tabla está vacía.
5. Postman envía `GET http://localhost:8080/hola`.
6. El **Controller** recibe la petición y llama al **Service**.
7. El **Service** llama al **Repository**.
8. El **Repository** ejecuta `SELECT * FROM mensaje` y devuelve el primer registro.
9. La respuesta sube por las capas hasta el **Controller**.
10. El **Controller** serializa el objeto a JSON y responde 200 OK con `{id:1,"texto":"Hola Mundo"}`.
11. Postman muestra el resultado.

Conclusiones

En este tutorial construimos desde cero una aplicación backend con Java, Spring Boot y PostgreSQL que:

- Se levanta con un único comando gracias a Docker Compose.
- Gestiona la base de datos automáticamente mediante el ORM Hibernate/JPA.
- Expone endpoints REST bien estructurados siguiendo la arquitectura de tres capas.
- Puede verificarse fácilmente desde Postman o `curl`.